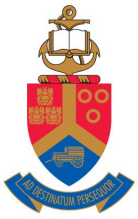

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS 216 Practical Assignment 3

- Date Issued: **16 March 2026**
- Date Due: **13 April 2026** before **08:00**
- Submission Procedure: **Upload to the web server (`wheatley`) + clickUP**
- This assignment consists of **6 tasks** for a total of **145 marks**.

1 Introduction

During this practical, you will be implementing your very own API that will later be used in your flight bookings website. You must ensure that it is efficient and secure from threats like SQL injection attacks. Furthermore, it should provide clear and concise error messages to the user. You will also implement basic signup and login functionality in your website.

After successful completion of this assignment you should also be able to create webpages in PHP which comply to the HTML5 and JavaScript Standards.

2 Constraints

1. You must complete this assignment individually.
2. You may ask the Teaching Assistants for help but they will not be able to give you the solutions.
3. You must produce all of the source files yourself; you may not use any tool to generate source files or fragments thereof automatically. (**This includes ChatGPT, Gemini etc.!!**)
4. Your assignment will be viewed using Chrome Web Browser so be sure to test your assignment in this browser. Nevertheless, you should take care to follow published standards and make sure that your assignment works in as many browsers as possible.
5. You may utilise any text editor or IDE, upon an OS of your choice, again, as long as you do not make use of any tools to generate your assignment. (**This includes ChatGPT, Gemini etc.!!**)
6. All written code should contain comments including your name, surname and student number at the top of each file.
7. Your assignment must work on the `Wheatley` web server, as you will be marked off there. However you are free and encouraged to **Develop** using a local tool like XAMPP and later move over to wheatley however it must work off wheatley during marking.
8. **You may use JavaScript and/or JQuery(You may not use Jquery for AJAX) for this however no other libraries are allowed (unless specified in a previous practical). You must use the PHP cURL library for the API you are developing.**
9. **Server-side scripting should be done using an Object-Oriented approach.**
10. **You must NOT use any features from PHP Version 8 or higher. Wheatley is on PHP version 7.3, therefore while version 8 features may work locally they will not work on Wheatley.**

3 Submission Instructions

You are required to upload all your source files (e.g. HTML5 documents, any images, etc.) to the **web server (`Wheatley`) and clickUP in a compressed (zip) archive**. Make sure that you test your submission to the web server thoroughly. All the menu items, links, buttons, *etc.* must work and all your images must load. Make sure that your practical assignment works on the web server before the deadline. No late submissions will be accepted, so make sure you upload in good time. The server will not be accepting any uploads and updates to files from the stipulated deadline time until the end of the marking week (Thursday at 3pm).

The deadline is on Sunday but we will allow you to upload until Monday 11am. After this NO more submissions will be accepted.

Note, `Wheatley` is currently available from anywhere. But do not rely that outside access from the UP network will always work as intended. You must therefore make sure that you `ftp` your assignment to the web server. Also make sure that you do this in good time. A snapshot of the web server will be taken just after the submission was due and only files in the snapshot will be marked.

Practicals are marked by demonstrating your practical to a tutor during the allocated marking weeks. **If you do not demonstrate your practical you will receive 0 for the practical**. You will only be marked in the practical session that you have booked on the cs portal, if you miss it, you will receive 0 (Unless special permissions have been granted by the lecturer. i.e. You were sick and able to provide a sick note).

NB: You must submit a README.txt file.
It should detail the following:

- How to use your website.
- Default login details (username and password) for a user you have in your Database (on Wheatley).
- Any functionality not implemented.
- Explanations for the password requirements, choice of hashing algorithm and generation of API keys.

4 Online resources

PHP Sessions - http://www.w3schools.com/php/php_sessions.asp

Timestamps - https://en.wikipedia.org/wiki/Unix_time

Cookie - https://www.w3schools.com/js/js_cookies.asp

PHP Headers - http://php.net/manual/en/function.header.php

PHP cURL - http://php.net/manual/en/book.curl.php

PHP GET - http://php.net/manual/en/reserved.variables.get.php

PHP POST - http://php.net/manual/en/reserved.variables.post.php

5 Rubric for marking

Database Setup	20
Include	2
Planes Table	3
Airports Table	4
User Table	6
ReadMe	5
Page Construction	15
Pages converted to PHP	3
New files (skeleton)	4
Navbar	5
Correct file structure	3
User Registration	15
Validation	10
API Call	3
Response	2
User Registration API	20
SQL	8
Validation	5
Security	5
Parameters	1
Response	1
Planes API	50
API Setup	4
Headers	2
Type	3
APIkey	3
Return	3
Fuzzy	5
Limit	4
Sort	8
Order	3
Search	15
Airport API	25
API Setup	4
Headers	3
Type	3
APIKey	3
Page	6
Search	6
Upload/Deductions	
Does not work on `Wheatley`	**-50**
Not uploaded to clickUP	**-145**
Not demoed	**-145**
SQL Injection Vulnerable	**-20**
Bonus	5
Total	**145**

6 Assignment Instructions

Task 1: Database Setup ..... (20 marks)

The first objective of this task is to **setup a database on Wheatley**. It is recommended that you do this through the use of phpMyAdmin. You can access it at `https://wheatley.cs.up.ac.za/phpmyadmin/`. The username is your student number and the password can be found in the *db_password* file found in your Wheatley root directory. Alternatively, you maybe use other tools to do this such as MySQL Workbench, or SQL command line.

Note: Your Database Names must be prefixed by your student number and an underscore (e.g. u12345678_products).

The next objective of this task is to import a given MySQL DB Dump. Along with the specification, a Database dump will be found on ClickUP. When imported correctly, it should result in a populated Planes and Airports table. This is the table you will use in a later task to pull data for your API.

For this practical you will need at least 1 table in your database apart from the Planes and airports table. This table will contain all your user information, so make sure to include the following fields:

- id
- name
- surname
- email
- password
- type (Refers to type of user: "Passenger", or "ATC")
- api_key

Task 2: Page Construction ..... (15 marks)

You will need to create the following pages as a skeleton for your project:

- `config.php`, `header.php`
- `login.php`, `logout.php` – used in Practical 4
- `signup.php` – see Task 3
- `api.php` **(this should be placed in the root of your Wheatley Directory)**

Note: These are the minimum required files and you are welcome to add anything extra you think is necessary. Each new PHP file (except api.php) should go in the same directory/folder as your existing webpages

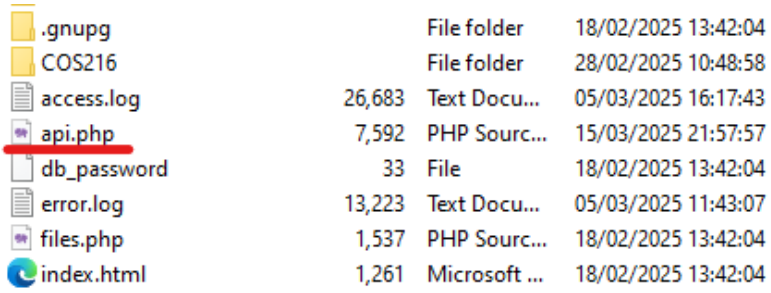

Image showing the correct location for your api

The objective of this task is firstly to make use of the ***include*** function to stitch pages together.

- Your header.php file should contain the navbar you made from your previous assignments (such that it can be repeated on each php page using the include function).

This means that every php page should get the header information from the header.php page. It is highly recommended that you include config.php in the header.php page as well. This config file serves as your database connection as well as global variables and other configurations you might need for your website.

This also means that all pages (HTML files) from the previous practical (with the exception of the launch page with links to all practicals) should be converted to PHP files.

The navigation for this entire project should be stored in a single file (header.php) and included in every page. Your navbar should also include links to login and register. If the user is already logged in, there should be no login and register links. Instead, the name of the user should be displayed on the website with a logout button (the functionality of logging in and out will be implemented in Practical 4). For this practical, your webpages **do not** need to make requests to your new API.

Task 3: User Registration ..... (15 marks)

This task focuses on the **signup page**. The goal is for the user to be able to enter various details into a form on the "signup page" and register an account on your flight booking website. Before the form data is submitted to the API, it must be processed by various validation functions using JavaScript, which check whether the information is correct or not (This is known as client-side validation). If it is valid, the user is added to the relevant table in your database by making a call to your API with the type "Register". (Please see the following task for information on the API)

You should complete the following functionality for this task:

- Create a signup form on the signup page (signup.php) with the following fields:
  - name
  - surname
  - email
  - password
  - type
- Before allowing the user to submit the form, client-side JavaScript must be used to check that all the fields are filled out correctly. This means that the email address should have an '@' symbol and the password should be longer than 8 characters, contain upper and lower case letters, at least one digit and one symbol

  (**NB: You need to explain why you think this is necessary in your ReadMe.txt file**). You must make use of **JS Regex** for this. No plugins or libraries are allowed. You may only copy a Regex string for the email from the web, but ensure you have the best one. Any other Regex needs to be done by yourself.

  Note: Using the HTML required attribute is not enough since one can easily change the type of input box.
- Make sure that the form submits the information via **POST** to your API after all validation (on the client-side) is successful.
- If the email address is already in use, the API will send back this information and then an error message will be displayed on the User Interface.

Task 4: User Registration API ..... (20 marks)

This task requires you to create a PHP API using **api.php** for your flight booking website in an Object-Oriented manner. Hence, you need to make use of PHP classes. **Hint:** take a look at how singletons work. You should save the API in a file called "**api.php**" and it should reside in your root folder.

NB: Your API should only produce/consume structured JSON data.
Also note that you do not yet need to exclusively use this API in your website yet, that will be done in PA4. You simply need to make sure the API itself is functional.

Your API must make use of an API key for each request, except for the "Register" type, in order to prevent unauthorized access or security attacks. The API key is simply a randomly generated key consisting of alphanumeric characters for an authorized party (more details in the next task). This task focuses on the **api.php** file and will be worked on in both PA3 and PA4. For this task, the goal is to code the API functionality that allows a user to register via the API. You will need to integrate this API with the User Interface you made in the previous task. It is vital that you do server-side validation, validating every input and returning back an appropriate response.

Your API should follow the following structure.
Request
url: wheatley.cs.up.ac.za/uXXXXXXXX/api.php - (URL to your PHP API)
method: POST - (HTTP Method)

JSON POST body

Parameter	Required	Description
type	Required	Method to identify what information is needed, consists of the following values: • ***Register*** - Tells the API what functionality to perform • More parameters will be introduced in the next task as well as PA4
name	Required	The name of the user registering
surname	Required	The surname of the user registering
email	Required	The email of the user registering
password	Required	The users password (see restrictions below)
user_type	Required	The type of User registering ("Customer", "Courier" or "Inventory Manager")

Request Example Here is an example of the post parameters you can use.

Example Post body:

```
{
        "type":"Register",
        "name": "Tony",
        "surname": "Stark",
        "email": "tony@starkindustries.com",
        "password": "LoveYou3000",
        "user_type": "ATC"
}
```

Response Object

Your API should return a structured JSON Object as a response. Here is what the response to the request example given above should be:

```
{
        "status": "success",
        "timestamp":"1679507636541",
        "data": [
                "apikey": "5c331d9c15d564d3d0de0f1f2937b92e"
        ]
}
```

You should complete the following functionality/restrictions for this task:

- You should perform input validation, that means, you need to check for missing, blank, or incorrect fields (This is known as server-side validation). This means that the email address should have an '@' symbol and the password should be longer than 8 characters, contain upper and lower case letters, at least one digit and one symbol

  (**NB: You need to explain why you think this is necessary in your README.txt file**). You must make use of **Regex** in PHP for this purpose. No plugins or libraries are allowed. You may only copy a regex string for the email from the web, but ensure you have the best one. Any other Regex needs to be done by yourself.

- The user should be validated (check if the user exists as well as validate the input fields). You may only use built-in PHP functionality for this, no libraries or frameworks are allowed. An appropriate error message should be returned by the API for any errors in the data, and a appropriate HTTP Status code should be returned as well. **Hint:** Make use of unique keys.
- Include the config.php file in the API in order to get the database connection to perform the necessary MySQL query to insert the user into the database table.
- It should go without saying that passwords should not be stored in plain text. You will need to **choose a Hashing algorithm**. You may **not** use Blowfish. (**NB: You will be evaluated on your choice so make sure to include this in your README.txt file**).
- You will need to **add salt** to your passwords to make them more secure. They should also be above 10 characters, as shorter salts are more susceptible to brute force attacks. Your salt should be dynamically created and not be a fixed string.
- If the email address is already in the table, the query should be ignored, and an error message returned.
- An **API key** needs to be generated once all the validation has been successful. The key should be a unique alpha-numeric string consisting of at least 10 characters. (**NB: You will be assessed on how you generated this key so make sure to include this in your README.txt file**). This API key should be shown to the user once it is created. The API key is used to interact with your API on the Types that require identification.

Task 5: Planes and Airports API ..... (75 marks)

You are required to implement functionality in your **api.php** to also support the GetAllPlanes and GetAllAirports types. Your API must only produce and consume **structured JSON data**.

For this section use `https://wheatley.cs.up.ac.za/api/doc.html#planes` and `https://wheatley.cs.up.ac.za/api/doc.html#airports` to see how your requests and responses should be structured.

If the request contains missing or invalid parameters your API should return:

```
{
 "status": "error",
 "timestamp": 1679391940921,
 "data": "Post parameters are missing"
}
```

Make sure to account for all other types of errors (Illegal input, invalid api key etc)

Security As we are dealing with SQL it is **CRUCIAL** your system is safe from SQL injection. (-10 if SQL injection is possible).

APIs are standalone and can be used through any interface that supports it (REST/SOAP). Make sure that your API works as you will need it for the remainder of the practicals (if you cannot get the API to work, as a last resort you may use mock data, however, that won't earn you many marks).

It is highly recommended that you use a tool like `https://postman.com` or `https://www.usebruno.com/` to test your API.

Task 6: Bonus ..... (5 marks)

For bonus marks, you can add extra functionality to the API by adding more security features and using the timestamp feature to do something cool like refreshing the data once a specific time has elapsed or caching data to make the website load faster. Be creative! Bonus marks will be capped to full marks.